



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Security & SSO: 3.5 → 4.0

Seven auth changes you cannot ignore

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0 Eight changes between your jobs and 4.0

Hibernate & JPA: 3.5 → 4.0 Ten data-layer changes, mostly the quiet kind

Security & SSO: 3.5 → 4.0 Seven auth changes you cannot ignore **THIS GUIDE**

Data & Messaging: 3.5 → 4.0 Nine changes across your stores and streams

Testing: 3.5 → 4.0 Seven test-suite breaks, one clean build away

Observability & Ops: 3.5 → 4.0 Six changes your dashboards will notice first

Jackson & JSON: 3.5 → 4.0 Nine JSON changes everyone hits

Web & HTTP: 3.5 → 4.0 Eight changes at the HTTP layer

Core Platform & Build: 3.5 → 4.0 Twenty platform changes underneath everything

The Big Jobs: L-Impact Changes The six changes that eat your sprint

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

7 CHANGES

5 × TIER 1 - WON'T BUILD

2 × TIER 2 - WON'T START

TIER	CHANGE	IMPACT	AUTO
1	ApacheDS Embedded LDAP Support Removed	M	•
1	OpenSAML 4 Support Removed	L	•
1	Security DSL Rewrite	L	Y
1	SimpDestinationMessageMatcher Removed	M	•
1	Spring Security Access API Moved to spring-security-access	M	•
2	OAuth 2.0 Password Grant Completely Removed	L	•
2	PKCE Mandatory for Confidential OAuth Clients	M	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

Security DSL Rewrite

THE HUMAN LIST

Read the card. Check your code. Decide.

ApacheDS Embedded LDAP Support Removed

OpenSAML 4 Support Removed

SimpDestinationMessageMatcher Removed

Spring Security Access API Moved to spring-security-access

OAuth 2.0 Password Grant Completely Removed

PKCE Mandatory for Confidential OAuth Clients

TIER 1

Tier 1 - Won't Build

5 cards

1. ApacheDS Embedded LDAP Support Removed
2. OpenSAML 4 Support Removed
3. Security DSL Rewrite
4. SimpDestinationMessageMatcher Removed
5. Spring Security Access API Moved to spring-security-access

ApacheDS Embedded LDAP Support Removed

Tier 1 - Won't Build

Security & SSO

Impact: M

ApacheDSContainer is removed in Spring Security 7.0. Use UnboundIdContainer instead.

WHAT CHANGED

`org.springframework.security.ldap.server.ApacheDSContainer` has been removed from Spring Security. The embedded LDAP server capability is now provided exclusively through

`org.springframework.security.ldap.server.UnboundIdContainer`.

WHY

ApacheDS development had stalled and its Jakarta EE compatibility lagged. UnboundID is actively maintained and provides equivalent functionality for embedded LDAP testing and development use cases.

SCOPE CHECK

Search for `ApacheDSContainer` and `apacheds-server-jndi` (or other ApacheDS artifacts) across Java sources and build files.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/ApacheDsUsage.java:[3,57]
error: package org.springframework.security.ldap.server does not contain
ApacheDSContainer
```

THE FIX

pom.xml — swap the embedded LDAP dependency

BEFORE

```
<dependency>
  <groupId>org.apache.directory.server</groupId>
  <artifactId>apacheds-server-jndi</artifactId>
  <scope>test</scope>
</dependency>
```

AFTER

```
<dependency>
  <groupId>com.unboundid</groupId>
  <artifactId>unboundid-ldapsdk</artifactId>
  <scope>test</scope>
</dependency>
```

Java config — replace the container class

BEFORE

```
import org.springframework.security.ldap.server.ApacheDSContainer;
// ...
new ApacheDSContainer("dc=springframework,dc=org", "classpath:users.ldif");
```


AFTER

```
import org.springframework.security.ldap.server.UnboundIdContainer;  
// ...  
new UnboundIdContainer("dc=springframework,dc=org", "classpath:users.ldif");
```

HOW TO FIX

Replace ApacheDSContainer with UnboundIdContainer.

Swap the dependency in your build file and update the import and instantiation. The constructor signature is the same: base DN and LDIF classpath location. The LDIF format is also compatible.

Use Spring Boot's embedded LDAP auto-configuration.

If you're only using embedded LDAP for tests, add `spring-boot-starter-ldap` and set `spring.ldap.embedded.base-dn` in test properties. Spring Boot will auto-configure UnboundID for you.

WATCH OUT

LDIF files written for ApacheDS should work with UnboundID, but verify any vendor-specific schema extensions you may have used.

FURTHER INFO

Both containers were available in 3.5; only UnboundID survives in 4.0.

spring-break module: `apacheds-ldap-removed` Spring Security 7 LDAP migration

OpenSAML 4 Support Removed

Tier 1 - Won't Build

Security & SSO

Impact: L

Spring Security 7 dropped OpenSAML 4 support and requires OpenSAML 5. Code referencing `OpenSaml4AuthenticationProvider` and related classes fails to compile on Boot 4.0.

WHAT CHANGED

Spring Security 7 removed the OpenSAML 4 compatibility classes and requires OpenSAML 5. The SAML support classes (`OpenSaml4AuthenticationProvider`, `OpenSaml4AuthenticationRequestResolver`, etc.) were replaced with OpenSAML 5 equivalents. OpenSAML 5 changed package structures and removed the marshaller/unmarshaller split.

WHY

OpenSAML 4 is no longer maintained. OpenSAML 5 brought significant API improvements, including a simplified object-provider model and better XML security defaults. Maintaining dual support across two major OpenSAML versions was unsustainable.

SCOPE CHECK

Search for `opensaml` in your Maven/Gradle files, and for `OpenSaml4` in Java code. If your application uses SAML 2.0 login (`saml2Login()` in your security config), you are affected. If you only use OAuth 2.0 / OIDC, this card does not apply.

WHAT YOU'LL SEE

```
$ mvn clean compile
[ERROR] COMPILATION ERROR :
[ERROR] SamlConfig.java:[4,63] cannot find symbol
    symbol: class OpenSaml4AuthenticationProvider
[INFO] BUILD FAILURE
---
The build stops at compile. The application never starts.
```

THE FIX

Maven dependency

BEFORE

```
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-api</artifactId>
  <version>4.3.2</version>
</dependency>
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-impl</artifactId>
  <version>4.3.2</version>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-api</artifactId>
  <version>5.1.2</version>
</dependency>
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-impl</artifactId>
  <version>5.1.2</version>
</dependency>
```

Java config — authentication provider

BEFORE

```
import org.springframework.security.saml2.provider.service.authentication
    .OpenSaml4AuthenticationProvider;

OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();
```

AFTER

```
import org.springframework.security.saml2.provider.service.authentication
    .OpenSaml5AuthenticationProvider;

OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();
```

Java config — custom response validator

BEFORE

```
OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
    Saml2ResponseValidatorResult result =
        OpenSaml4AuthenticationProvider
            .createDefaultResponseValidator()
            .convert(responseToken);
```

AFTER

```
OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
    Saml2ResponseValidatorResult result =
        OpenSaml5AuthenticationProvider
            .createDefaultResponseValidator()
            .convert(responseToken);
```

HOW TO FIX

Upgrade OpenSAML dependencies to 5.x.

Update your OpenSAML dependencies to version 5.1.x. If you use Spring Boot's dependency management, the managed version will already be OpenSAML 5 in Spring Boot 4.0. Remove any explicit version overrides pinning to 4.x.

Rename Spring Security SAML classes.

Replace all references to `OpenSam14*` classes with their `OpenSam15*` equivalents. The main classes are `OpenSam15AuthenticationProvider` and `OpenSam15AuthenticationRequestResolver`.

Review custom SAML processing code.

If you directly use OpenSAML APIs (marshallers, unmarshallers, builders), review the OpenSAML 5 migration guide. The marshaller/unmarshaller split was removed and the object provider model changed.

WATCH OUT

OpenSAML 5 requires Java 17+ and has different XML processing requirements. Verify your XML security libraries are compatible.

The Shibboleth Maven repository is still required for OpenSAML 5 artifacts. Make sure your build still has that repository configured.

If you have custom `Sam12AuthenticationRequestContext` processing or assertion decryption, those APIs may have changed signatures in the OpenSAML 5 upgrade.

FURTHER INFO

The change comes from Spring Security 7.0, which Spring Boot 4.0 ships. Affects spring-boot-starter-security consumers using SAML 2.0 login.

[Spring-Break Demo](#) [Spring Security 7 migration](#) [Spring Boot 4.0 Migration Guide](#)

Security DSL Rewrite

Tier 1 - Won't Build

Security & SSO

Impact: L

🔄 OpenRewrite

WHAT CHANGED

Spring Security 7.0 (shipped with Boot 4.0) removed the deprecated `authorizeRequests()` method: use `authorizeHttpRequests()` instead. `antMatchers()`, `mvcMatchers()`, and `regexMatchers()` are replaced by `requestMatchers()`. The `.and()` chaining method is gone; the lambda DSL takes its place.

WHY

The lambda DSL is more readable and avoids a class of configuration bugs where `.and()` chains silently reset the security context. The new `requestMatchers()` API auto-detects whether to use Ant or MVC matching based on your classpath.

SCOPE CHECK

Search for `authorizeRequests()`, `antMatchers()`, `mvcMatchers()`, `.and()` in security config classes, and `.csrf().disable()`. Every `SecurityFilterChain` bean needs reviewing.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/SecurityConfig.java:[22,14]
  error: cannot find symbol
    symbol:   method authorizeRequests()
    location: variable http of type HttpSecurity
[ERROR] /src/main/java/com/example/SecurityConfig.java:[23,18]
  error: cannot find symbol
    symbol:   method antMatchers(java.lang.String)
[ERROR] /src/main/java/com/example/SecurityConfig.java:[28,10]
  error: cannot find symbol
    symbol:   method and()
```

THE FIX

Security filter chain — before

BEFORE

```
http
    .authorizeRequests()
        .antMatchers("/public/**").permitAll()
        .antMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    .and()
    .formLogin()
        .loginPage("/login")
    .and()
    .logout()
        .logoutSuccessUrl("/");
```

AFTER

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/public/**").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    )
    .formLogin(form -> form
        .loginPage("/login")
    )
    .logout(logout -> logout
        .logoutSuccessUrl("/")
    );
```

CSRF configuration

BEFORE

```
http.csrf().disable();
```

AFTER

```
http.csrf(csrf -> csrf.disable());
```

HOW TO FIX

Rewrite to lambda DSL.

Replace every `.and()` chain with a lambda-based configuration block. Each security concern (authorization, form login, logout, CSRF) gets its own lambda. See the [Spring Security 7 migration guide](#).

Replace matchers.

Change `antMatchers()` and `mvcMatchers()` to `requestMatchers()`. The new method auto-selects the matching strategy. `regexMatchers()` becomes `requestMatchers(new RegexRequestMatcher(...))`.

VERIFY

App starts and security filter chain initialises without errors

WATCH OUT

The `.and()` removal is the most disruptive part. A long chain with 5+ `.and()` calls means restructuring the whole method. Review the result carefully: it's easy to nest a lambda inside the wrong block.

`requestMatchers()` auto-detects MVC vs Ant matching. If you have Spring MVC on the classpath, it uses MVC matching (which is stricter). If this changes behaviour, use `AntPathRequestMatcher` explicitly.

FURTHER INFO

Driven by Spring Security 7.0. The old DSL was deprecated in Security 5.7/6.0; the lambda DSL has been the recommended replacement since Security 5.2. Affects every spring-boot-starter-security consumer. See also: `websecurity-adaptor-removed`, `auth-default-deny`.

spring-break module: `security-removed-apis` [Spring Security 7 migration guide](#)

SimpDestinationMessageMatcher Removed

Tier 1 - Won't Build

Security & SSO

Impact: M

WHAT CHANGED

`org.springframework.security.messaging.util.matcher.SimpDestinationMessageMatcher` has been removed. Spring Security 7.0 replaces the old `AbstractSecurityWebSocketMessageBrokerConfigurer` / matcher-based approach with `MessageMatcherDelegatingAuthorizationManager`.

WHY

The old configurer and matcher classes were deprecated in Security 6.x. 7.0 removed them, completing the move to `AuthorizationManager`.

SCOPE CHECK

Search for

`SimpDestinationMessageMatcher` and `AbstractSecurityWebSocketMessageBrokerConfigurer`. Both are removed. Only applications using STOMP over WebSocket with Spring Security are affected.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/SimpDestUsage.java:[3,66]
error: package org.springframework.security.messaging.util.matcher
does not contain SimpDestinationMessageMatcher
```

THE FIX

Old — matcher-based WebSocket security config

BEFORE

```
import org.springframework.security.messaging.util.matcher.SimpDestinationMessageMatcher;
// ...
new SimpDestinationMessageMatcher("/topic/**");
```

New — `AuthorizationManager`-based config

ADDED

```
import org.springframework.security.messaging.access.intercept.MessageMatcherDelegatingAuthorizationManager;
// ...
MessageMatcherDelegatingAuthorizationManager.builder()
    .simpDestMatchers("/topic/**").authenticated()
    .anyMessage().denyAll()
    .build();
```

HOW TO FIX

Migrate to `MessageMatcherDelegatingAuthorizationManager`.

Replace any use of `SimpDestinationMessageMatcher` and `AbstractSecurityWebSocketMessageBrokerConfigurer` with a `MessageMatcherDelegatingAuthorizationManager` bean registered on your message broker. See the Spring Security 7 messaging migration guide for the full configuration pattern.

WATCH OUT

The new `AuthorizationManager` must be wired into the message broker configuration explicitly. There is no auto-migration path: the configuration structure changes substantially.

FURTHER INFO

Part of Spring Security 7.0's WebSocket security overhaul: the same `AuthorizationManager` pattern now applies across HTTP, method, and message security.

spring-break module: `simpdest-message-matcher-removed` [Spring Security 7 messaging migration](#)

Spring Security Access API Moved to spring-security-access

Tier 1 - Won't Build

Security & SSO

Impact: M

AccessDecisionManager and related legacy authorisation classes moved to a separate spring-security-access module. Add it or migrate to AuthorizationManager.

WHAT CHANGED

AccessDecisionManager ,
AccessDecisionVoter ,
@EnableGlobalMethodSecurity ,
and related legacy authorisation classes have been moved from spring-security-core to a new standalone spring-security-access artifact.

WHY

Spring Security 7.0 makes AuthorizationManager the sole supported authorisation API. The legacy Access API survives for migration, isolated in its own module so finished migrations don't carry the dead weight.

SCOPE CHECK

Search for
AccessDecisionManager ,
AccessDecisionVoter ,
@EnableGlobalMethodSecurity ,
and
org.springframework.security.access imports across all sources.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/AccessApiUsage.java:[3,48]
error: package org.springframework.security.access does not exist
```

THE FIX

pom.xml — add legacy module if needed

ADDED

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-access</artifactId>
</dependency>
```

Preferred: migrate to AuthorizationManager

BEFORE

```
import org.springframework.security.access.AccessDecisionManager;
// implements AccessDecisionManager
```

AFTER

```
import org.springframework.security.authorization.AuthorizationManager;
// implements AuthorizationManager<RequestAuthorizationContext>
```

HOW TO FIX

Add spring-security-access (short-term fix).

Add spring-security-access as an explicit dependency. This restores compilation without any logic changes and buys time to do the full migration.

Migrate to `AuthorizationManager` (recommended).

Replace `AccessDecisionManager` / `AccessDecisionVoter` implementations with `AuthorizationManager` .
Replace `@EnableGlobalMethodSecurity` with `@EnableMethodSecurity` . The Spring Security migration guide has step-by-step instructions.

WATCH OUT

`@EnableGlobalMethodSecurity` is also in the legacy module. If you use method security, you need either the legacy module or to migrate to

`@EnableMethodSecurity` before removing it.

FURTHER INFO

Driven by Spring Security 7.0; the [spring.io](#) announcement in the footer covers the full background.

spring-break module: [spring-security-access-relocated](#) [Spring Security access API moves](#)

TIER 2

Tier 2 - Won't Start

2 cards

1. OAuth 2.0 Password Grant Completely Removed

2. PKCE Mandatory for Confidential OAuth Clients

OAuth 2.0 Password Grant Completely Removed

Tier 2 - Won't Start

Security & SSO

Impact: L

Spring Security 7 removed the OAuth 2.0 Resource Owner Password Credentials grant. Apps that authenticate with it fail at startup or return 401s at runtime.

WHAT CHANGED

The password grant, deprecated in Spring Security 5.x, is gone in Spring Security 7 (shipped with Spring Boot 4.0). The

`ClientRegistration` builder rejects `authorization-grant-type: password`, and the `ResourceOwnerPasswordResourceDetails` class no longer exists.

WHY

The password grant hands user credentials directly to the client application, defeating the point of OAuth delegation. The OAuth 2.1 specification (RFC 9700) formally removed it; Spring Security followed.

SCOPE CHECK

Search for `authorization-grant-type: password` in YAML/properties files and `AuthorizationGrantType.PASSWORD` in Java code. Also search for `ResourceOwnerPasswordResourceDetails` which was the OAuth 2.0 password grant helper class. Any hit means that authentication flow needs redesigning.

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'securityFilterChain'
...
Caused by: java.lang.IllegalArgumentException:
  The grant type 'password' is not supported.
  Supported grant types are: [authorization_code, client_credentials,
  refresh_token, urn:ietf:params:oauth:grant-type:device_code,
  urn:ietf:params:oauth:grant-type:token-exchange]
  at org.springframework.security.oauth2.client.registration.ClientRegistration$Builder.build(...)
---
APPLICATION FAILED TO START
```

THE FIX

application.yml — client registration

BEFORE

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
            authorization-grant-type: password
            client-id: my-client
            client-secret: secret
        provider:
          my-api:
            token-uri: https://auth.example.com/oauth/token
```

AFTER

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
            authorization-grant-type: authorization_code
            client-id: my-client
            client-secret: secret
            scope: openid,profile
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          my-api:
            issuer-uri: https://auth.example.com
```

Java configuration — programmatic registration

BEFORE

```
ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.PASSWORD)
    .tokenUri("https://auth.example.com/oauth/token")
    .clientId("my-client")
    .clientSecret("secret")
    .build();
```

AFTER

```
ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
    .authorizationUri("https://auth.example.com/authorize")
    .tokenUri("https://auth.example.com/oauth/token")
    .redirectUri("{baseUrl}/login/oauth2/code/{registrationId}")
    .clientId("my-client")
    .clientSecret("secret")
    .build();
```

HOW TO FIX

Migrate to Authorization Code + PKCE.

Replace the password grant with Authorization Code flow. For browser-based apps, use Authorization Code with PKCE. This is the recommended approach and requires changes to both the client application and possibly the authorization server configuration.

Use Client Credentials for service-to-service.

If the password grant was used for machine-to-machine communication (no actual user), switch to the Client Credentials grant. This does not require user interaction.

Custom token exchange as a last resort.

If you must support legacy systems that send username/password, implement a custom token exchange endpoint on your authorization server that accepts credentials and issues tokens. This keeps the anti-pattern in one controlled place while you plan a proper migration.

VERIFY

Token endpoint returns 200 for auth code flow (not password grant)

WATCH OUT

The grant is gone from the OAuth 2.1 specification itself, so your authorization server (Keycloak, Auth0, Okta) may drop support too. Check both sides.

Mobile apps that used password grant to avoid browser redirects should migrate to Authorization Code with PKCE, which modern mobile OAuth SDKs support natively.

Integration tests that authenticate by posting username/password to the token endpoint will break. Update test helpers to use a test-specific grant flow or mock the security context directly.

FURTHER INFO

Removal tracked in [spring-security#6003](#). Affects `spring-boot-starter-oauth2-client` consumers. See also: `pkce-mandatory`.

[spring-break module: oauth-password-grant-removed](#) [OAuth Password Grant removal issue](#) [Spring Security 7 migration](#)

[Spring Boot 4.0 Migration Guide](#)

PKCE Mandatory for Confidential OAuth Clients

Tier 2 - Won't Start

Security & SSO

Impact: M

Spring Security 7 enforces PKCE for all OAuth 2.0 clients, including confidential clients. Authorization requests without a code_challenge are rejected by the authorization server.

WHAT CHANGED

Spring Security 7 now sends PKCE parameters (`code_challenge` and `code_challenge_method`) on every Authorization Code request, for both public and confidential clients. If the authorization server requires PKCE but the client was not sending it (pre-upgrade behaviour for confidential clients), or if the authorization server rejects unexpected PKCE parameters, the flow breaks.

WHY

OAuth 2.1 (RFC 9700) mandates PKCE for all clients, public and confidential alike, because PKCE prevents authorization code interception attacks regardless of client type. Spring Security aligned with that requirement.

SCOPE CHECK

Check every OAuth 2.0 client registration in your application. For each one, verify that the corresponding authorization server accepts PKCE parameters. If you use Spring Authorization Server, it already supports PKCE. If you use a third-party provider, check their documentation.

WHAT YOU'LL SEE

```
[oauth2-client] o.s.s.oauth2.client.web.OAuth2LoginAuthenticationFilter:
  Authentication request failed:
org.springframework.security.oauth2.core.OAuth2AuthenticationException:
  [invalid_request] PKCE code_challenge is required for this client.
  at org.springframework.security.oauth2.client.oidc.authentication
    .OidcAuthorizationCodeAuthenticationProvider.authenticate(...)
  ---
HTTP 302 → /login?error=invalid_request
---
Browser shows: "OAuth2 Error: [invalid_request]
PKCE code_challenge is required for this client."
```

THE FIX

application.yml — no changes needed on client side

BEFORE

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-app:
            authorization-grant-type: authorization_code
            client-id: my-confidential-client
            client-secret: ${CLIENT_SECRET}
            scope: openid,profile
```

AFTER

```
# Client config stays the same – Spring Security 7 adds PKCE automatically.
# But your authorization server MUST accept PKCE parameters.
spring:
  security:
    oauth2:
      client:
        registration:
          my-app:
            authorization-grant-type: authorization_code
            client-id: my-confidential-client
            client-secret: ${CLIENT_SECRET}
            scope: openid,profile
```

Keycloak realm config — enable PKCE for confidential client

BEFORE

```
"clientId": "my-confidential-client",
"publicClient": false,
"pkceCodeChallengeMethod": ""
```

AFTER

```
"clientId": "my-confidential-client",
"publicClient": false,
"pkceCodeChallengeMethod": "S256"
```

Disable PKCE enforcement if auth server can't support it (temporary)

BEFORE

```
// default PKCE behaviour
```

AFTER

```
@Bean
public OAuth2AuthorizationRequestResolver pkceResolver(
    ClientRegistrationRepository repo) {
    DefaultOAuth2AuthorizationRequestResolver resolver =
        new DefaultOAuth2AuthorizationRequestResolver(
            repo, "/oauth2/authorization");
    // Temporary: remove when auth server supports PKCE
    resolver.setAuthorizationRequestCustomizer(
        OAuth2AuthorizationRequestCustomizers.withoutPkce());
    return resolver;
}
```

HOW TO FIX

Enable PKCE on your authorization server.

Configure your authorization server (Keycloak, Auth0, Okta, Azure AD) to accept PKCE `code_challenge` parameters for confidential clients. Most modern providers support this; it may need enabling per client. Use

`s256` as the challenge method.

Verify no auth server rejects extra parameters.

Some older or custom authorization servers reject unknown parameters. If yours rejects `code_challenge`, you need to update the auth server first, or temporarily disable PKCE enforcement on the Spring Security side using a custom resolver.

Disable PKCE temporarily (not recommended).

If you cannot update the authorization server immediately, register a custom

`OAuth2AuthorizationRequestResolver` that strips PKCE parameters. Treat this as technical debt and plan the auth server update.

VERIFY

OAuth2 login completes with PKCE challenge in the auth request

WATCH OUT

The failure often appears only in the browser redirect flow. Automated tests that mock the OAuth exchange may not trigger the error. Test with a real authorization server.

Azure AD and some ADFS versions may need tenant-level configuration changes to allow PKCE for confidential clients. Check your identity provider's documentation.

If you use Spring Authorization Server as your auth server and it is also being upgraded, both sides get PKCE support automatically. The risk is when the client (Spring Boot 4) and the auth server (third-party or older version) are upgraded independently.

FURTHER INFO

PKCE enforcement was tracked in [spring-security#16391](#). Affects `spring-boot-starter-oauth2-client` consumers using the `authorization_code` grant. See also: `oauth-password-grant`.

[spring-break module: pkce-mandatory](#) [PKCE enforcement issue](#) [Spring Security 7 migration](#)

[Spring Boot 4.0 Migration Guide](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki

OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4

HeroDevs NES for Spring • herodevs.com/support/spring-nes

The test suite behind this book • github.com/spoole167/spring-break